

OOPS

OOPS - stands for Object Oriented Programming System

oops allows us to create our own objects that have methods and attributes.

useful for organizing code well

these methods act as functions that uses information of objects as well as object itself to return results or change the current object.

In general, oops allows us to create code that is repeatable and organized. For much larger of code scripts, functions arent enough for organization and repeatability.

Commonly repeatable objects and tasks can be defined with OOP to create code that is more usable.

✓ What are Classes and Objects?

Classes and objects are the two core concepts in object-oriented programming.

A class defines what an object should look like, and an object is created based on that class. For example:

Class Objects

Fruit Apple, Banana, Mango

Car Volvo, Audi, Toyota

When you create an object from a class, it inherits all the variables and functions defined inside that class.

```
# syntax
class NameOfClass():
    def __init__(self,param1,param2):
        # __init__ is a special method called automatically
        # when a new object (instance) of the class is created

        # self refers to the current object being created

        # param1 value passed during object creation
        # is stored in the instance variable self.param1
    print(self.param1)
```

✓ class

Almost everything in Python is an object, with its properties and methods.

A Class is like an object constructor, or a "blueprint" for creating objects.

To create a class, use the keyword class:

```
# Create a class named MyClass, with a property named x:
class MyClass:
    x = 4
```

```
Myclass
```

```
__main__.Myclass
```

▼ Create Object

Now we can use the class named MyClass to create objects:

```
# Create an object named p1, and print the value of x:  
p1 = MyClass()  
print(p1)
```

```
<__main__.Myclass object at 0x7b212f566150>
```

```
print(p1.x)
```

```
4
```

```
# Delete Objects  
# You can delete objects by using the del keyword:  
del p1
```

▼ Multiple Objects

You can create multiple objects from the same class:

```
p1 = MyClass()  
p2 = MyClass()  
p3 = MyClass()
```

```
print(p1.x)  
print(p2.x)  
print(p3.x)
```

```
4  
4  
4
```

Each object is independent and has its own copy of the class properties.

The pass Statement

class definitions cannot be empty, but if you for some reason have a class definition with no content, put in the pass statement to avoid getting an error.

OOPS real content

Object is an real world entity, everything in the world and python is object Class is a blueprint of object, we use class to design object. We can create as many as possible objects in class. Most of the projects and frameworks are done in OOPs

OOPS is a programming paradigm which means an approach or method or style to solve a problem or task. there are different names for it we use this for programming efficiency and time complexity Python follows 3 programming paradigms they are:

1. Procedural PP
2. Functional PP
3. OOPS PP This is why python called multiprogramming paradigm

Procedural pp means dividing the problem into procedures and completing it Functional PP means works on using the logic and evaluate we can not solve data centric problems using above paradigms because those 2 mainly focuses on computation. Alen kay introduced oopp in 1966 for simulation programs.

OOPS follows bottom-up approach: every task is divided into components to solve or find solutions.OOPS is based on object concept.

Object

Everything is considered as object in python and world.An object is an instance of the class.it means object is a copy and product camme from class Every object has property and behaviour. Any thing that has property and behaviour is considered as object,ex: human has properties like name,DOB,height, weight and gender and behaviour like work you are doing. We represent object as follows:

1.properties: ATTRIBUTES or VARIABLES

2.Behaviours: METHODS(FUNCTIONS) or PROCEDURES

both exist in object, due to this we can get data security, accessibility and code structure. To create an object we must first declare a class.As we said before class is a blueprint of object. whenever you check datatype of anything in python, it first shows class.

why we need OOPS?

To solve real world complex projects, to do that we need extensibility featured programming paradigm.so oops follows bottom-up approach, through this we can extend our limit to solve. Features and advatages of OOPS:

DATA SECURITY

ALLOWS PARALLEL PROGRAMMING(while working on same class others can work on their objects.) also called as MODULARITY

REUSABILITY

COST & TIME REDUCTION

SOFTWARE MAINTAINANCE IS EASY

FASTER PROBLEM OR CODE FIXING

OOPS CONCEPTS

There are totally 6 concepts in oops they are:

Class

object

Inheritance

Polymorphism

Abstraction

Encapsulation

class: it is a collection of objects

If we create methods and varriables to the class, then every object created in the class contain all the variables and methods that exist in that class.

whenever we want to perform a task, then we look at object properties and methods, then we create a class with similar objects which have similar properties and behaviours.

```
#synatx to ccreate class:
class ClassName(): # classname should follow camel casing means every word starts with capital letter
# class is a keyword we should define first to create class.
# class name is an identifier, className is name of the class we want to create and we can name it as we want
# there will be set of statements in the class with indentation.(4 wihtespaces indentation)
statements
.
.
.
.
statements n
```

```
File "/tmp/ipykernel_14106/3450063895.py", line 6
    statements
    ^
IndentationError: expected an indented block after class definition on line 2
```

```
# lets create our first class
class SampleClass():
    pass
```

```
# if we declare variables in class then they are called as attributes.
# lets pass attributes to class and access them
class Sample():
    atr = 1
    atr2 = 2
```

```
# how to access attributes or methods in the class
# classname.attribute
# classname.methodname
# lets access
Sample.atr
```

1

```
# how to call a class?
# classname(), thats it
Sample()
```

```
<__main__.Sample at 0x7c44e05b3560>
```

▼ creating object:

we can create object but no use if we dont store object in a variable

```
# creating object and storing it in a variable by calling a class
object = Sample()
obj = Sample()
```

```
# accessing attributes in the class through object
object.atr
obj.atr2
```

2

```
# changing or deleting attributes of the object
object.atr = 3
```

```
object.atr
```

3

```
# but changing in object does not affect in obj lets see
obj.atr
Sample.atr
# by this we can conclude that no object has the scope to modify or delete attributes in other objects and class.
```

1

```
# but w can modify attributes in the class and that updates in all objects lets see
Sample.atr = 5
print(object.atr)
print(obj.atr)
print(Sample.atr)
Sample.atr
```

3

5

```
5
5
```

```
# HOW TO CREATE METHOD IN THE CLASS?
# it is just like creating a function but here we have to mention self word inside method
class m():
    def meth(self):
        print("hello")
```

```
m()
```

```
<__main__.m at 0x7c44cfe22b40>
```

```
o = m()
# accessing methods in objects
o.meth()
```

```
hello
```

Let us look at **init** method

whenever we call class, then this init method will automatically executes unlike other methods. If we want attributes to exist in an object from the beginning, we define and assign them in **init**, and pass their values when creating the object. so when u create object, then that data will store in that object.

we call this init method as constructor syntax for init method **def init(self, parameter1, parameter2, ... parameterN):**

```
self.pm1 = pm1(pm=parameter)
self.pm2 = pm2
self.pmn = pmn
```

there self parameter in every method in class as first parameter.

```
# lets create init method
class init:
    def __init__(self):
        print("this method executes automaically when we call class")
```

```
object1 = init()
# here it shows outpt without calling method unlike other methods
```

```
this method executes automaically when we call class
```

```
# we initialize an object using this init method
class BankAccount:
    def __init__(self, accountNo, accountName, ifscCode, balance):
        self.accountNo = accountNo
        self.accountName = accountName
        self.ifscCode = ifscCode
        self.balance = balance

    def display(self):
        print(self.accountNo, self.accountName, self.ifscCode, self.balance)

# object creation
object1 = BankAccount(12345678910, 'dhoni', 'MPT10010122', 10000)

# method call
object1.display()

12345678910 dhoni MPT10010122 10000
```

self parameter

self is a current object. if we call a class then this current object store in the self. so here in above while calling class we havent given any argument to the self parameter because it stores object. so here is the qsn, why we have mentioned self.variable = variable this is

because whatever we have given in init method, that has to be stored in object, so we give it as self.variable name the very common way we store attribute value in object is this:

object.attribute = value

we can name it as self.a = account no instead of self.accountno but object stores account value

self refers to the current object. We don't pass it manually because Python automatically sends the object to it. Inside **init**, we use self.variable = value to store data in the object so it can be used later. Without self, the values won't persist in the object.

every method has this self, because it is the reference to object in the class.

we can name self as we like, we can replace it with this, key, or anything you want.

what if there is self parameter,

```
class BankAccount:
    def __init__(self, acc_number, acc_name, ifsc_code, balance=0):
        self.acc_number = acc_number
        self.acc_name = acc_name
        self.ifsc_code = ifsc_code
        self.balance = balance

    # Method to deposit money
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited ₹{amount}. New balance: ₹{self.balance}")
        else:
            print("Invalid deposit amount")

    # Method to withdraw money
    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount")
        elif amount > self.balance:
            print("Insufficient balance")
        else:
            self.balance -= amount
            print(f"Withdrawn ₹{amount}. Remaining balance: ₹{self.balance}")

    # Method to check balance
    def check_balance(self):
        print(f"Account Balance: ₹{self.balance}")

    # Display account details
    def display_details(self):
        print("\n--- Account Details ---")
        print(f"Account Number : {self.acc_number}")
        print(f"Account Name : {self.acc_name}")
        print(f"IFSC Code : {self.ifsc_code}")
        print(f"Balance : ₹{self.balance}")

# ----- Usage Example -----
acc1 = BankAccount("1234567890", "Naveen", "SBIN0001234", 1000)

acc1.display_details()
acc1.deposit(500)
acc1.withdraw(300)
acc1.check_balance()
```

```
--- Account Details ---
Account Number : 1234567890
Account Name : Naveen
IFSC Code : SBIN0001234
Balance : ₹1000
Deposited ₹500. New balance: ₹1500
Withdrawn ₹300. Remaining balance: ₹1200
Account Balance: ₹1200
```

▼ INHERITANCE

inheritance is one of the pillar concepts in oops Property of using methods n attributes in a class from created class called inheritance. Created class is Parent or super class and the class we gonna create is called child or sub class. inheriting class is simple:

def ChildClass(Parentclass):

automatically all methods and attributes will be accessed to child class new def: creating a class from already created class real world ex: children getting genes and qualities from parents

There are 5 types of Inheritance, which are supported by python, they are:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

Double-click (or enter) to edit

```
# Single inheritance
# creating a class using single class or using single class is called single inheritance
class parent():
    def method1(self):
        print("parent class")
```

```
class child(parent):
    def method2(self):
        print("child class")
```

```
object = child() # created object for class child
object.method1() # accessing method from parent class to object via child class
object.method2() # accessing method from child class
obj = parent()
obj.method2()
```

```
parent class
child class
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_1200/539951796.py in <cell line: 0>()
      3 object.method2() # accessing method from child class
      4 obj = parent()
----> 5 obj.method2()

AttributeError: 'parent' object has no attribute 'method2'
```

we can access parent class methods but parent class can not access child class methods without calling child class, if we try then python throws an error

Multiple Inheritance

this 2nd kind of inheritance in oops here we can access or create class from mutiple classes like a kid getting qualities from both mom n dad

```
class mom():
    def method1(self):
        print("mom class")
```

```
class dad():
    def method2(self):
        print('dad class')
```

```
class kid(mom, dad): # inheritance from multiple classes
    def kidm(self):
        print("im kid")
```

```
kids = kid()
kids.method2()
kids.method1()
```

```
dad class
mom class
```

Method resolution order

using this method we decide which class method should execute first based on the order we have given in the class() it shows the order we have given

if we want to access any method to use, then that object looks at this order kid, mom, dad.

this will apply to every inheritance

```
print(kid.__mro__)
```

```
(<class '__main__.kid'>, <class '__main__.mom'>, <class '__main__.dad'>, <class 'object'>)
```

```
# lets another example
class mom():
    def method1(self):
        print("mom class")
```

```
class dad():
    def method1(self):
        print('dad class')
```

```
class kid(dad, mom):
    def kk(self):
        print("hello kid")
```

```
obj = kid()
obj.method1()
```

```
dad class
```

look at the above codes, there is same method in both classes method1, but while passing classes to kid we have given dad class first so based on that it called that method.

multi-level inheritance

property of inheriting from multi levels, grand parent-parent-child

creating class from already created class which is also created by a super class

```
class a():
    def m1(self):
        print("m1")
```

```
class b(a):
    def m2(self):
        print("m2")
```

```
class c(b):
    def c(self):
        print("ABC")
```

```
O = c()
O.m1()
```



```
O.m2()
O.c()
```

```
m1
m2
ABC
```

▼ Hierarchical Inheritance

Hierarchical inheritance is a type of inheritance where **multiple child classes inherit from a single parent class**.

It means one base (parent) class is shared by more than one derived (child) class, and each child can have its own additional features while still using the common properties and methods of the parent.

Example

It's like one parent having multiple children — all children inherit some common traits, but each child can also have unique characteristics.

```
# Parent class
class Animal:
    def eat(self):
        print("Animal eats food")

# Child class 1
class Dog(Animal):
    def bark(self):
        print("Dog barks")

# Child class 2
class Cat(Animal):
    def meow(self):
        print("Cat meows")

# Creating objects
dog = Dog()
cat = Cat()

# Using inherited method
dog.eat() # from Animal
cat.eat() # from Animal

# Using their own methods
dog.bark()
cat.meow()
```

```
Animal eats food
Animal eats food
Dog barks
Cat meows
```

▼ Hybrid inheritance

combo of more than one inheritance, single+multli like that

```
# Base class
class Animal:
    def eat(self):
        print("Animal eats food")

# Derived from Animal (hierarchical part)
class Dog(Animal):
    def bark(self):
        print("Dog barks")

class Cat(Animal):
    def meow(self):
        print("Cat meows")

# Another base class
class Pet:
    def play(self):
        print("Pet plays")
```

```
# Hybrid class (multiple inheritance + hierarchical)
class Puppy(Dog, Pet):
    def weep(self):
        print("Puppy weeps")

# Object
p = Puppy()

# Accessing methods
p.eat()    # from Animal
p.bark()   # from Dog
p.play()   # from Pet
p.weep()   # own method
```

```
Animal eats food
Dog barks
Pet plays
Puppy weeps
```

method call cheyakunda direct vadali ante use init method in classes 🤔🤔

Advantages of Inheritance

1. Code reusability
2. Reduced redundancy
3. Easy maintenance
4. Extensibility
5. Better organization
6. Supports polymorphism

mainly to reduce duplication, to maintain rlnships bw objects

Disadvantages of Inheritance

1. tightly coupled- spuer and sub classes are independent of each other, modification in one class can affect other as well

```
# code with duplication
class Benz:
    def start(self):
        print("Car starts")

    def stop(self):
        print("Car stops")

class BMW:
    def start(self):
        print("Car starts")

    def stop(self):
        print("Car stops")
```

```
# removing duplication using inheritance
# Parent class
class Car:
    def start(self):
        print("Car starts")

    def stop(self):
        print("Car stops")

# Child classes
class Benz(Car):
    def luxury(self):
        print("Benz has luxury features")

class BMW(Car):
    def sport(self):
        print("BMW has sporty features")

# Usage
```

```

b = Benz()
a = BMW()

b.start()
a.start()

b.luxury()
a.sport()

```

```

Car starts
Car starts
Benz has luxury features
BMW has sporty features

```

Method Overloading

If a class have multiple methods with same name and different parameters and size then it is called method overloading.

```

# same method with different types of parameters
class a:
    def m(int, int):
        pass
    def m(str, str):
        pass
    def m(float, float):
        pass

```

```

File "/tmp/ipykernel_1200/2484563835.py", line 3
    def m(int, int):
            ^
SyntaxError: duplicate argument 'int' in function definition

```

```

# same method with different no of parameters
class b:
    def m(int):
        pass
    def m(int, int):
        pass
    def m(int, int, int):
        pass

```

```

File "/tmp/ipykernel_1200/2306938827.py", line 5
    def m(int, int):
            ^
SyntaxError: duplicate argument 'int' in function definition

```

```

# same method with different data types and differentt no of parameters
class c:
    def m(int, int):
        pass
    def m(str, str):
        pass
    def m(float, float, float):
        pass
# in this if you pass values in object like 1,2 then first method calls out, if it is "o","k" then 2nd method calls out, it wor

```

```

File "/tmp/ipykernel_1200/4133246984.py", line 3
    def m(int, int):
            ^
SyntaxError: duplicate argument 'int' in function definition

```

Instead of creating `add_two()`, `add_three()`, `add_four()` separately, you just have one `add()` method that handles all cases. You pass however many numbers you need, and the same method takes care of the addition. It saves you from writing repetitive code and keeps things simple and organized.

▼ advantages of method overloading

1. Flexibility
2. Readability
3. easy to see and reduce complexity

python doesnt support method overloading because py is dynamically typed language, it wont check data type as we wont declare datatype in py

```
# lets see example
class ad:
    def add(self,a,b):
        print("ok")
    def add(self,a,b,c):
        print("pl")
```

```
o = ad()
o.add(1,2)
```

```
-----
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1200/623558409.py in <cell line: 0>()
      1 o = ad()
----> 2 o.add(1,2)

TypeError: ad.add() missing 1 required positional argument: 'c'
```

we assumed Python would treat both methods as separate versions (like overloading). It didn't.

When you defined the method the second time, Python **replaced the first one completely**. It doesn't store multiple versions based on parameters—only the **last definition exists**.

So your class ended up with just one method that expects **three arguments**.

Then you called it with only **two arguments**, which is why Python threw an error saying one argument is missing.

That's the whole issue:

- No overloading in Python
- Last method wins
- Argument mismatch causes the error

then how do we use method overlaoding?

simple- by installing multipledispatch.

using dispatch method we can use method overloading in python

```
!pip install multipledispatch
# installing multipledispatch module
```

```
Requirement already satisfied: multipledispatch in /usr/local/lib/python3.12/dist-packages (1.0.0)
```

```
from multipledispatch import dispatch # importing dispatch method from module
```

```
import multipledispatch
class A:
    @dispatch(int, int)
    def add(self, a, b):
        return a + b
    @dispatch(int, int, int)
    def add(self, a, b, c):
        return a + b + c
    @dispatch(float, float)
    def add(self, a, b):
        return a + b
```

```
mo = A()
mo.add(1,2)
mo.add(1,2,3)
# this is how you can handle method overlaoding
```

6

```
class b:
    def add(self, *args):
        sum = type(args[0])()
        for i in args:
            sum += i
        return sum
```

```
obj = b()
obj.add(1,2,3)
```

6

```
obj.add(1.2,2.3)
```

3.5

▼ METHOD OVERRIDING

superclass & superclass has same method . If we access the method in subclass then, only subclass method is accessed without accessing superclass method.

Rules for Method overriding :

1. Super class & Subclass must be present (Inheritance must be done between two classes)
2. Declare two classes with same method & same parameters
3. Logic must be different in methods
4. Method overriding will done when we access the same method in subclass. If we access in superclass then method will not override

```
class parent:
    def method(self):
        print("im parent")
```

```
class child(parent):
    def method(self):
        print("im kid")
```

```
obj = child()
obj.method() # here sub class method overrides super class method
```

im kid

then how do we handle method overriding?

1st way: parentclassname.method(self,arguements if any)

```
# lets see example
class sup:
    def method(self):
        print("nen ra AI ni ra")
```

```
class sub(sup):
    def method(self):
        sup.method(self) #1st way to handle
        print("nanne ra antav ra ")
```

```
objection = sub()
objection.method()
```

```
nen ra AI ni ra
nanne ra antav ra
```

```
# 2nd way: super().methodname(arguments)
class sup:
    def method(self):
        print("nen ra AI ni ra")
class sub(sup):
    def method(self):
        super().method() #2nd way to handle
        print("nanne ra antav ra ")
```

```
objection = sub()
objection.method()
```

```
nen ra AI ni ra
nanne ra antav ra
```

```
# handling method overriding init method
# 1st way
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # call parent init
        self.age = age
```

```
child = Child("Alice", 5)
print(child.name)
print(child.age)
```

```
Alice
5
```

▼ POLYMORPHISM

polymorphism is a greek word, poly – many, morphism – forms

so polymorphism means having many forms, for ex: a person can have many forms like a person can be student, child, brother, nephew etc..

it is an ability to do more than one task

The same method name can perform different actions depending on the object or input.

```
def fun(*args):
    if args:
        start = type(args[0])()
        for i in args:
            start += i
        return start
    # this function is an ex for polymorphism
```

```
a = fun(2,3)
a
```

```
5
```

```
fun(1,3.4)
```

```
4.4
```

```
class a:
    def fun(self):
        print("gun")
class b:
    def fun(self):
        print("fun")
```

```
def poly(obj):
    obj.fun()
```

```
ob = a()
o = b()
poly(o)
poly(ob)
```

```
fun
gun
```

```
# Here, a single function works with different objects, demonstrating polymorphism
```

✓ POLYMORPHISM USING METHOD OVERLOADING

Same method doing task for different data types and parameters, that means method overloading is also performing polymorphism so method overloading also supports polymorphism

```
# lets see code
from multipledispatch import dispatch
class A:
    @dispatch(int, int)
    def a(self, a, b):
        return a+b
class B:
    @dispatch(str, str)
    def a(self,b,v):
        return b+v
```

```
obj1 = A()
obj2 = B()
print(obj1.a(1,2))
obj2.a("h","n")
# here same method called "a" performed addition and string concatenation, same method but different forms
```

```
3
'hn'
```

✓ polymorphism in method overriding

having same method in super and sub class and sub class method overrides super class method though it has many forms, but it choose right form among them

```
class sup():
    def add(self):
        print("this is super")
```

```
class sub(sup):
    def add(self):
        sup.add(self)
        print("im not super class")
```

```
obj1 = sub()
obj1.add()
```

```
this is super
im not super class
```

```
# calling only super class method using sub class method
obj = sub()
sup.add(obj)
```

```
this is super
```

there are two types of polymorphisms

1. compile time polymorphism or static polymorphism-

if polymorphism done during compiling then it is static one ex: method overloading.

2. run time polymorphism or dynamic polymorphism

if its done during run time then its dynamic one, ex: method overriding. this decides during run time so..we also call it dynamic binding.

ENCAPSULATION

Combining or grouping related things into a single unit, ex: burger

Encapsulation hides implementation logic and allows only to use.(abstraction).

class is an example which groups related data, methods and objects together.

Reasons for encapsulation:

1. Organized code and clean
2. prevents data from accidental modification.
3. Abstraction
4. data hiding.(in classes using access modifiers we secure data so that nobody modify them.)
5. Modularity(individual development)

data hiding + abstraction = Encapsulation

ACCESS MODIFIERS

Sometimes we have to limit the access for data and methods for data privacy and security.

this encapsulation provides access modifiers for data security and privacy

we have 3 access specifiers, they are:

1. public access specifiers
2. protected access specifiers
3. private access specifiers

but in python private, protected data can be accessed same as public data

1. Public access specifiers

If data is declared in public access specifier we can access data

. In same class

. By object

. we can access from subclass & Access by subclass object

we can use it anywhere as it is public one, by default every data is public

```
# public code
class public:
    publicdata = 10
    def pub(self):
        print(self.publicdata)
# class 2
```



```
class two(public):
    def method(self):
        print(self.publicdata)
```

```
obj1 = public()
obj1.pub()
obj2 = two()
obj2.method()
```

```
10
10
```

2. protected access specifiers

If data is declared as protected, it can be accessed within the same class and its derived (child) classes. even with objects also we can not access.

put `_` [single underscore] before data & method to declare as protected

```
class protected:
    _protecteddata = 10 # marking it as protected
    def _protected(self):
        print(self._protecteddata)
class child(protected):
    def method(self):
        print(self._protecteddata)
```

```
obj = protected()
obj._protected()
obj1 = child()
obj1.method()
print(obj._protecteddata) # accessing variable via object
```

```
10
10
10
```

3. private access modifiers

If data is declared as private access specifier . It can be accessed only in that class.

put `__` [double underscore] before data & method to declare as private access modifier or specifier or member

```
class Private:
    __privatedata = 10 # private variable

    def __privatemethod(self):
        print(self.__privatedata)

class Public(Private):
    def method(self):
        # accessing private variable using name mangling
        print(self._Private__privatedata)
```

```
obj2 = private()
obj2.__privatemethod()
obj1 = public()
obj1.method()
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_811/2479331893.py in <cell line: 0>()
      1 obj2 = private()
----> 2 obj2.__privatemethod()
      3 obj1 = public()
      4 obj1.method()

AttributeError: 'private' object has no attribute '__privatemethod'
```

here in above we got error because we can not access data through objects, only through the class, but before we have mentioned that we can access private as public, then how? so here is the qsn, why do we give data in modifiers? ans: to avoid name collisions we use public, private and protected in python. based on priority of data we use different modifiers.

private data is more sensitive so developers handle it carefully and rarely access it

NAME MANGLING

NameMangling - Declare data or method with atleast 2 leading underscores and atmost 1 trailing underscore. Then it will replace to `_classNameName` by the interpreter

`_data`

HERE 2 underscores is leading and single one is trailing.

Purpose of name mangling (in Python):

1. Avoid accidental access

It prevents other classes or developers from accidentally using or modifying internal variables.

2. Avoid name conflicts in inheritance

If a child class defines the same variable name, mangling ensures it doesn't clash with the parent's private variable.

3. Indicate "internal use"

It signals: "this is meant to be private, don't touch it directly."

```
class A:
    __a = 1
    b_ = 2
    __c__ = 3
    ____d_ = 4
    ___e___ = 5
print(dir(A))
# here we are checking which variable name will get mangled, so to do that we have to pass this class to dir method
```

```
['_A____d_', '_A_a', '__c__', '___e___', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__']
```

Which variables get name mangled?

Only variables that:

start with two underscores (__)

do NOT end with two underscores

✓ Mangled:

`__a` → `_A_a`

`____d` → `A____d` (starts with __, so it counts)

✗ Not mangled:

`b_` → normal (no leading __)

`_c` → ends with __ → treated like special name → no mangling

`e` → ends with __ → no mangling

```
# now coming back to private access modifier,
class A:
    __a = 1
    def getpvt(self):
        print(self.__a)
    # here we can access data only in the created class
    # also we cannot access private member with object and it throws an error
```

```
obj1 = A()
```

```
obj1.getpvt()
obj1.__a_
```

```
1
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_811/2688425119.py in <cell line: 0>()
      1 obj1 = A()
      2 obj1.getpvt()
----> 3 obj1.__a_

AttributeError: 'A' object has no attribute '__a_'
```

- as above code shows error, thats due to private access modifier, we can not access private member in sub class

```
# now lets fix that and access private member through class object and sub class using name mangling. dir() method provides whi
class A:
    __a_ = 1
    def getpvt(self):
        print(self.__a_)
class B(A):
    def method(self):
        print(self._A__a_)
```

```
# creating objects for both classes and accessing via objects and sub class
object1 = A()
object1._A__a_
object2 = B()
object2.method()
object2._A__a_
# here we go
```

```
1
1
```

Data hiding

Data hiding is one of the key concepts in OOP. It refers to restricting direct access to data so that it cannot be accessed or modified from outside the class.

It is used to protect sensitive data and ensure that it is accessed only through controlled methods.

Data hiding is achieved using encapsulation. if we want to hide the data in python, firstly we have to declare data as private member, with this data can be accessed within the class, no outside classes access it.

to perform that we have to put __[double underscore] before data or method to declare as private member

```
class bank:
    __balance = 49.00
    def getBalance(self):
        print(self.__balance)
class API(bank):
    def method(self):
        print(self.__balance)
```

```
obj = API()
obj.method()
obj.getBalance()
```

```

-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_7483/118665834.py in <cell line: 0>()
      1 obj = API()
----> 2 obj.method()

```

```

# lets fix above code
class bank():
    # declaring private member
    __bankBalance = 49.00
    def getBalance(self):
        print(self.bankBalance)

# deriving new class from bank class
class API(bank):
    def method(self):
        print(self.__bank__bankBalance)

```

```

# creating object for derived class
object1 = API()
object1.method()
object1.__bank__bankBalance

```

```

49.0
49.0

```

Always access private data through class methods instead of directly through objects, to prevent unintended modification and maintain control over the data.

this private wont restrict developers to use data but gives responsibility to handle it with care

✓ ABSTRACTION

Abstraction is the process of hiding unnecessary implementation details and showing only the essential features to the user.

It helps in handling complexity by focusing on what to do rather than how it is done.

Example:

A bank ATM allows you to withdraw money, check balance, etc., without exposing the internal working of the system.

We implement abstraction using abstract classes in python OOPS

Abstract class

if a class contain one or more than one abstract method then that class is called abstract class

abstract method

if we declare method without implementation then its abstract method

```

# this is abstract method
class abs:
    def method(self): # just declared but did not implement it
        pass

```

here class abs is abstract class and we cant create objects for this kind of classes, even if we try py throws an error

Start coding or [generate](#) with AI.